

Tarea C — Recuperación léxica contra semántica latente sobre un corpus mínimo

MMIA 6013 · Taller 03 v2 · reporte.pdf

Objetivo

Comparar, sobre un corpus de 10 documentos y 6 consultas con un documento relevante cada una, la recuperación léxica TF-IDF (línea base) contra la semántica latente (LSA), evaluando ambas con Hit@1, Hit@3 y MRR; identificar en qué consultas falla cada método analizando el solape de términos entre consulta y documento relevante, y discutir qué aportaría un recuperador denso en el caso fallido (tarea-c-recuperacion-lexica-lsa.pdf).

Método

Los 10 documentos (d01–d10) y las 6 consultas (q1–q6) con sus juicios de relevancia se escriben literalmente en el script.

- **TF-IDF:** `TfidfVectorizer()` con todos los parámetros por defecto (minúsculas, tokens de 2+ caracteres, sin stop words, unigramas, norma l2), ajustado sobre los 10 documentos (vocabulario: 105 términos); las consultas se transforman con el mismo vectorizador y se ordenan los 10 documentos por similitud coseno descendente para cada consulta.
- **LSA:** `TruncatedSVD(n_components=4, random_state=0)` ajustado sobre la matriz TF-IDF de los documentos; documentos y consultas se proyectan a 4 dimensiones con ese mismo SVD, se normalizan con norma L2 (`Normalizer`) y se evalúa la similitud coseno en el espacio latente.
- **Métricas** sobre las 6 consultas: Hit@1 (relevante en el puesto 1), Hit@3 (relevante en el top 3) y MRR (media de 1/rango del relevante) (`s2-rag-y-vector-search.md`).

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
from sklearn.metrics.pairwise import cosine_similarity

DOCS = {"d01": "El mecanismo de atención pondera cada token según su similitud ...",
        ...} # d01..d10, textos literales del enunciado
QUERIES = {"q1": ("¿Qué función de ranking léxica pondera la frecuencia de términos?", "d0
        ...} # q1..q6 con su documento relevante
ids = list(DOCS)

vec = TfidfVectorizer() # todos los parámetros por defecto
X = vec.fit_transform(DOCS.values()) # matriz TF-IDF de documentos (10 x 105)
Q = vec.transform([t for t, _ in QUERIES.values()])

svd = TruncatedSVD(n_components=4, random_state=0)
Xl = Normalizer().fit_transform(svd.fit_transform(X)) # documentos latentes
Ql = Normalizer().fit_transform(svd.transform(Q)) # consultas latentes

def evaluar(S): # S: similitudes (6 consultas x 10 docs)
    rangos = {}
    for i, (qid, (_, rel)) in enumerate(QUERIES.items()):
        orden = [ids[j] for j in np.argsort(-S[i])]
        rangos[qid] = orden.index(rel) + 1 # rango 1-based del relevante
    return (np.mean([r == 1 for r in rangos.values()]),
            np.mean([r <= 3 for r in rangos.values()]),
            np.mean([1 / r for r in rangos.values()]), rangos)

print("TF-IDF:", evaluar(cosine_similarity(Q, X)))
print("LSA : ", evaluar(cosine_similarity(Ql, Xl)))
```

Resultados

Comparación de métodos (Parte 2). q1–q6 = rango (1-based) del documento relevante de cada consulta.

Método	Hit@1	Hit@3	MRR	q1	q2	q3	q4	q5	q6
TF-IDF	0.833	0.833	0.861	1	1	1	1	6	1
LSA	0.667	0.833	0.750	1	1	1	3	6	1

TF-IDF coloca el relevante en el puesto 1 en 5 de 6 consultas; LSA pierde el primer puesto en q4 (rango 3) y ambos métodos fallan q5 (rango 6).

Análisis por consulta (Parte 3). Top-3 obtenido y términos compartidos entre cada consulta y su documento relevante (* = stop word; en negrita, los fallos: relevante fuera del top 3).

Consulta	Método	Rango	Top-3	Términos compartidos
q1	TF-IDF	1	d04, d01, d06	de*, frecuencia, función, la*, léxica, pondera, ranking, términos
q1	LSA	1	d04, d01, d06	de*, frecuencia, función, la*, léxica, pondera, ranking, términos
q2	TF-IDF	1	d03, d01, d02	al*, fragmentos, prompt
q2	LSA	1	d03, d07, d09	al*, fragmentos, prompt
q3	TF-IDF	1	d08, d06, d04	bajo, de*, entrena, matrices, rango
q3	LSA	1	d08, d02, d06	bajo, de*, entrena, matrices, rango
q4	TF-IDF	1	d05, d01, d04	compara, la*, orientación, vectores
q4	LSA	3	d01, d10, d05	compara, la*, orientación, vectores
q5	TF-IDF	6	d03, d06, d04	la* (solo stop words)
q5	LSA	6	d04, d01, d03	la* (solo stop words)
q6	TF-IDF	1	d02, d06, d03	autoatención, capas, con*, feed, forward
q6	LSA	1	d02, d08, d10	autoatención, capas, con*, feed, forward

El único fallo de ambos métodos es **q5** («¿Qué hace que el modelo sea más determinista al elegir la siguiente palabra?», relevante d09 sobre temperatura y softmax): el solape consulta–d09 está formado solo por la stop word «la» (peso TF-IDF 0.2440 en la consulta y 0.3238 en el documento; el vectorizador por defecto no elimina stop words, pero su poder discriminativo es mínimo). Sin términos de contenido compartidos, la similitud de d09 es 0.0790 y queda en el puesto 6, mientras d03 encabeza con 0.3107 compartiendo con la consulta solo palabras generales como «modelo».

Discusión

LSA no corrige el fallo. Con solo 10 documentos, los 4 componentes latentes explican 0.39534217090869533 de la varianza: la «semántica» inducida por SVD procede de co-ocurrencias del propio corpus y no puede asociar «determinista» con «temperatura/softmax» si esos conceptos nunca coocurren. q5 permanece en el rango 6 (top-3 LSA: d04, d01, d03) y, además, LSA degrada q4 del rango 1 al 3, con lo que Hit@1 baja de 0.833 a 0.667.

Parte 4 — ¿Por qué un recuperador denso resolvería q5 mejor? Porque los embeddings de un modelo entrenado codifican sinonimia y paráfrasis aprendidas de grandes corpus: «más determinista al elegir la siguiente palabra» quedaría cerca de «valores bajos concentran la probabilidad» (temperatura y softmax en d09) aunque no compartan ninguna palabra de contenido. La recuperación léxica es exacta con términos técnicos, siglas y nombres propios, pero no entiende sinónimos ni paráfrasis (s2-rag-y-vector-search.md), y la LSA de 4 dimensiones entrenada sobre 10 documentos tampoco cierra esa brecha. **Costo frente a TF-IDF:** el recuperador denso exige un modelo preentrenado e inferencia para embeber consultas y documentos, mayor cómputo y memoria (índice de vectores densos), menor interpretabilidad (no hay pesos por término que expliquen el ranking) y riesgo de desajuste de dominio; TF-IDF no requiere entrenamiento, es barato e interpretable (s2-rag-y-vector-search.md). En la práctica ambos se combinan en búsqueda híbrida con fusión de rankings (RRF) (s2-rag-y-vector-search.md).